

DAY 4 RESULTS

4.2
/ 10

Toughest day yet — but very fixable

Q5 and Q6 were genuinely good. The rest had recurring, fixable issues: WHERE vs HAVING, copy-paste without adapting, wrong JOIN conditions, and EXCEPTS typo. No conceptual collapse — just execution errors.

Full marks

1

Partial

5

Failed

4

vs Day 3

-1.1

4-DAY TREND

Day 1	6.0
Day 2	6.9
Day 3	5.3
Day 4	4.2

QUESTION-BY-QUESTION BREAKDOWN

Q1 — Recursive CTE (retry)

★★★★☆ 4/10

Progress: You wrote the anchor correctly and used UNION ALL. Two critical gaps: (1) The recursive part never JOINS back to emp_heir — without that, it's just querying the raw table again, creating an infinite loop. (2) No path concatenation — the question specifically asked for 'Alice > Bob > Carol'. The SELECT at the end also needs a JOIN to get manager names.

```
WITH RECURSIVE emp_heir AS (  
  -- Anchor: CEO only  
  SELECT id, name, manager_id,  
         1 AS level,  
         name AS path  
  FROM org  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- Recursive: JOIN back to the CTE  
  SELECT o.id, o.name, o.manager_id,  
         h.level + 1,  
         h.path || ' > ' || o.name  
  FROM org o  
  JOIN emp_heir h ON o.manager_id = h.id -- ← this join was missing  
)  
SELECT e.id, e.name,  
       m.name AS manager_name,  
       e.level, e.path  
FROM emp_heir e  
LEFT JOIN org m ON e.manager_id = m.id  
ORDER BY e.level, e.id;
```

Q2 — MERGE / UPSERT (retry)

★★★★☆ 3/10

Wrong table names throughout. You copied the MERGE skeleton from Day 3's answer key but never swapped in the actual tables from this question — still says `target_orders`, `source_orders`, `order_id`, `amount`. The MERGE structure itself is correct, which tells me you understand the pattern. You just didn't adapt it.

```
MERGE INTO master_prices t
USING source_prices s
ON t.product_id = s.product_id
WHEN MATCHED THEN
  UPDATE SET t.price = s.price,
            t.last_updated = CURRENT_TIMESTAMP
WHEN NOT MATCHED THEN
  INSERT (product_id, product_name, price, last_updated)
  VALUES (s.product_id, s.product_name, s.price, CURRENT_TIMESTAMP);
```

Q3 — INSERT INTO ... SELECT (retry)

★★★★☆ 3/10

Good: No `VALUES()` wrapper this time — that's the Day 2 fix landing. But only 1 of 4 requirements met: Missing `UPPER(status)`, missing `amount > 0` filter, and missing `order_date <= CURRENT_DATE` filter. Read every bullet in the question before writing.

```
INSERT INTO clean_orders (order_id, customer_id, amount, status, order_date)
SELECT order_id, customer_id, amount,
       UPPER(status),
       order_date
FROM orders_staging
WHERE status IS NOT NULL
      AND amount > 0
      AND order_date <= CURRENT_DATE;
```

Q4 — GROUP BY correctness (retry)

★★★★☆ 6/10

Correct columns, correct `COUNT(DISTINCT)`, good instinct. One recurring bug: `WHERE total_revenue > 10000` — you cannot filter on an aggregate alias in `WHERE`. Aggregates are computed after `WHERE` runs. Use `HAVING`. Also use `EXTRACT(YEAR FROM order_date)` instead of `YEAR()` for cross-dialect safety.

```
SELECT category,
       EXTRACT(YEAR FROM order_date) AS d_year,
       SUM(amount) AS total_revenue,
       COUNT(DISTINCT customer_id) AS distinct_customers,
       ROUND(AVG(amount), 2) AS avg_order_value
FROM orders
GROUP BY category, EXTRACT(YEAR FROM order_date)
HAVING SUM(amount) > 10000 -- ← not WHERE
ORDER BY d_year, category;
```

Q5 — ROWS vs RANGE

★★★★☆ 9/10

Excellent. Both queries written correctly in a single `SELECT` — smart approach. Explanation about gaps is on the right track. To be precise: `ROWS` counts physical rows so gaps in dates don't matter to it; `RANGE` with an `INTERVAL` boundary includes all rows

within that calendar distance, so if a date is missing, it looks back further in time to find 6 calendar days of data. They diverge specifically when `sale_date` has duplicate values or missing days.

Q6 — Correlated subquery

★★★★☆ 6/10

Deviation calculation is correct. Two issues: (1) Question asked for a correlated subquery — you used a window function instead (still valid SQL, but shows you defaulted to window functions when the technique was specified). (2) `ORDER BY percent_dev_sal` `DESC` misses negative deviations — someone 30% below average has higher absolute deviation than someone 10% above. Use `ABS()`. Also the derived table is missing an alias.

```
SELECT name, salary,
ROUND(
  (salary - (SELECT AVG(salary) FROM employees e2
              WHERE e2.department = e1.department))
  / (SELECT AVG(salary) FROM employees e2
      WHERE e2.department = e1.department) * 100, 1
) AS pct_deviation
FROM employees e1
ORDER BY ABS(
  salary - (SELECT AVG(salary) FROM employees e2
            WHERE e2.department = e1.department)
) DESC;
```

Q7 — CROSS JOIN

★★★★☆ 3/10

`CROSS JOIN + LEFT JOIN` is the right pattern. `JOIN` condition is broken: `ON r.region_id=s.region_id OR s.categories_id=c.categories_id` — using `OR` here means a row matches if EITHER key matches, producing a cartesian explosion instead of a filtered join. You need `AND`. Also typo: `categories_id` should be `category_id`.

```
SELECT r.region_name, c.category_name,
COALESCE(SUM(s.amount), 0) AS total_sales
FROM regions r
CROSS JOIN categories c
LEFT JOIN sales s
ON s.region_id = r.region_id
AND s.category_id = c.category_id -- AND, not OR
GROUP BY r.region_id, r.region_name, c.category_id, c.category_name
ORDER BY r.region_name, c.category_name;
```

Q8 — EXCEPT / anti-join

★★★★☆ 2/10

Only wrote one approach, and it's logically broken. A single row can't have `YEAR = 2022 AND YEAR NOT IN (2023, 2024)` simultaneously in one scan — you needed two separate sets joined via subquery logic. Neither the `EXCEPT` nor the correct anti-join pattern was written.

```
-- EXCEPT approach:
SELECT p.product_id, p.product_name
FROM products p
JOIN sales s ON p.product_id = s.product_id
WHERE EXTRACT(YEAR FROM s.sale_date) = 2022
EXCEPT
```

```
SELECT p.product_id, p.product_name
FROM products p
JOIN sales s ON p.product_id = s.product_id
WHERE EXTRACT(YEAR FROM s.sale_date) IN (2023, 2024);
```

-- Anti-join approach:

```
SELECT DISTINCT p.product_id, p.product_name
FROM products p
JOIN sales s2022 ON p.product_id = s2022.product_id
  AND EXTRACT(YEAR FROM s2022.sale_date) = 2022
LEFT JOIN sales s_later ON p.product_id = s_later.product_id
  AND EXTRACT(YEAR FROM s_later.sale_date) IN (2023, 2024)
WHERE s_later.product_id IS NULL;
```

Q9 — Incremental load / CDC diff

★★★★☆ 2/10

Three issues: (1) `EXCEPTS` is not valid SQL — it's `EXCEPT`. (2) You wrote three disconnected queries; the question asked for a single unified report labelling each row as `NEW` / `CHANGED` / `DELETED`. (3) The logic for each case is also wrong — `EXCEPT` doesn't take a `WHERE` clause. Use `FULL OUTER JOIN` to compare both sides at once.

```
SELECT id, 'NEW' AS change_type
FROM source s
WHERE NOT EXISTS (SELECT 1 FROM target t WHERE t.id = s.id)

UNION ALL

SELECT s.id, 'CHANGED'
FROM source s
JOIN target t ON s.id = t.id
WHERE s.name != t.name OR s.email != t.email

UNION ALL

SELECT id, 'DELETED'
FROM target t
WHERE NOT EXISTS (SELECT 1 FROM source s WHERE s.id = t.id);
```

Q10 — Query optimisation

★★★★☆ 5/10

Good: Moving the subquery to a `JOIN` via CTE is a real improvement and shows optimisation thinking. But the two biggest problems remain unfixed: `YEAR(order_date)` and `UPPER(status)` both wrap columns in functions — this prevents the database from using indexes on those columns. The fix is to avoid calling functions on the indexed column side.

```
-- Optimised:
SELECT o.*
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE c.country = 'India'
  AND o.order_date >= '2023-01-01'    -- range scan, index-friendly
  AND o.order_date < '2024-01-01'    -- no function on column
```

```
AND o.status = 'completed';      -- store status lowercase,  
                                -- or use a generated column
```

Honest assessment — the downward trend needs attention

D1: 6.0 → D2: 6.9 → D3: 5.3 → D4: 4.2. Three separate problems are compounding:

1. **Copy-paste without adapting** (Q2 — wrong table names from the answer key).
2. **WHERE vs HAVING** — this has appeared in Q2 (Day 1), Q4 (Day 4). It must stop.
3. **Not reading all requirements** — Q3 had 4 conditions, you checked 1.

These are discipline issues, not knowledge gaps. Slow down, re-read the question after writing, and verify each requirement is met before submitting.

Non-negotiable focus for Day 5

1. **WHERE vs HAVING** — write it on paper: WHERE filters rows before grouping; HAVING filters groups after. Never filter an aggregate in WHERE.
2. **JOIN ON always uses =, never OR** for multi-key joins — use AND.
3. **Read every bullet** in the question and tick each one off in your query before submitting.
4. **Adapt, don't copy** — if you know the pattern, write it fresh with the correct table names.
5. **EXCEPT not EXCEPTS** — one word, no S.

Start Day 5 ↗